

Implement Client Script & UI Policy (Incident)

TEAM DETAILS

Team ID: SWTID-2026-8514

Team Size: 5

Team Leader: DHIVYA DHARSHINI R

Team Member: DIVYALAKSHMI R

Team Member: GAYATHRI S

Team Member: GUNASUNDARI SIVAKUMAR

Team Member: HARINI M

1. Introduction

Client Scripts and UI Policies are used in ServiceNow to control form behavior and improve the user experience. A Client Script can perform actions on the form when a specified event occurs, while a UI Policy can make fields mandatory, visible, or read-only based on conditions. This project demonstrates their use in the Incident module.

2. Problem Statement

Incident forms may contain fields that need to be controlled depending on the incident information entered by a user. Without dynamic form behavior, users may enter incomplete information or access fields that are not relevant to the current incident. The objective is to apply Client Script and UI Policy rules so that the Incident form responds appropriately to defined conditions.

3. Objectives

- Implement a Client Script for Incident form behavior.
- Implement a UI Policy to control Incident form fields.
- Make fields mandatory, visible, or read-only based on conditions.
- Validate and improve data entry on the Incident form.
- Provide a consistent and user-friendly Incident form.

4. Scope

This project focuses on client-side form behavior in the Incident module. It can be adapted for incident fields such as priority, impact, urgency, assignment group, category, and other fields where form behavior depends on selected values.

5. Technologies / Components

Component	Purpose
Client Script	Controls client-side form behavior
UI Policy	Controls field visibility, mandatory state, and read-only state
Incident Form	Provides the fields and user interface
Conditions / Events	Determine when the rule should be applied

6. Functional Requirement

The system must respond to selected Incident field values and apply the configured form behavior. For example, when a defined condition is met, the UI Policy can make a field mandatory or read-only, while the Client Script can perform additional client-side actions.

7. Client Script and UI Policy Logic

- IF the configured Incident condition is satisfied → Apply the required UI Policy behavior.
- IF the Client Script event occurs → Execute the defined client-side action.
- IF the condition is not satisfied → Return the form to the appropriate normal state.

8. Sample Configuration

Client Script:

Use the appropriate Client Script type such as onChange, onLoad, or onSubmit depending on the required Incident form event.

UI Policy:

Define a condition on the Incident form and configure the affected field to become mandatory, visible, or read-only as required.

9. Implementation Steps

- Open the ServiceNow Incident module and identify the required fields.
- Create a Client Script for the required Incident form event.
- Select the appropriate table, script type, and condition.
- Create a UI Policy for the required Incident form condition.
- Configure the affected fields using UI Policy Actions.
- Save and test the form with different field values.
- Verify that the Client Script and UI Policy work together as expected.

10. Test Cases

Test Case	Condition / Action	Expected Result
TC-01	Condition satisfied	Configured field behavior applied
TC-02	Condition not satisfied	Normal field behavior maintained
TC-03	Client Script event occurs	Defined client-side action executes
TC-04	Invalid / incomplete input	Form validation or configured behavior is applied

11. Expected Output

When a user works with an Incident record, the configured Client Script and UI Policy should apply the required form behavior. Fields should become mandatory, visible, or read-only according to the defined conditions, and the Client Script should execute the specified client-side action when its event occurs.

12. Advantages

- Improves Incident form usability.
- Reduces incomplete or incorrect data entry.
- Provides dynamic field control.
- Makes business rules easier to apply on the form.
- Can be customized for different Incident requirements.

13. Limitations

The exact behavior depends on the ServiceNow configuration, table settings, conditions, scripts, UI Policy Actions, and other client-side rules. Client-side controls should therefore be tested with the complete Incident form configuration.

14. Conclusion

The project demonstrates how Client Scripts and UI Policies can be used together to control the behavior of an Incident form. By applying conditions dynamically, the system can guide users to enter appropriate information and provide a more controlled and consistent Incident management experience.

15. Future Enhancements

- Add multiple condition-based UI Policies.
- Add more Client Scripts for different Incident events.
- Use advanced validation for Incident data.
- Add automated test cases for form behavior.
- Extend the solution to other ServiceNow modules.